Topics in Software Reliability

Material drawn from [Somerville, Mancoridis]

© SERG

What is Software Reliability?

- **<u>A Formal Definition:</u>** Reliability is the probability of failure-free operation of a system over a specified time within a specified environment for a specified purpose.

© SERG

What is Software Reliability?

- **<u>An Informal definition:</u>** Reliability is a measure of how closely a system matches its stated specification.
- **<u>Another Informal Definition:</u>** Reliability is a measure of how well the users perceive a system provides the required services.

© SERG

Software Reliability

- It is difficult to define the term objectively.
- Difficult to measure user expectations,
- Difficult to measure environmental factors.
- It's not enough to consider simple failure rate:
  - Not all failures are created equal; some have much more serious consequences.
  - Might be able to recover from some failures reasonably.

© SERG

Failures and Faults

- A **failure** corresponds to unexpected run-time behavior observed by a user of the software.
- A **fault** is a static software characteristic which causes a failure to occur.

© SERG

Failures and Faults (Cont'd)

- Not every fault causes a failure:
  - Code that is "mostly" correct.
  - Dead or infrequently-used code.
  - Faults that depend on a set of circumstances to occur.
- If a tree falls ...
  - If a user doesn't notice wrong behavior, is it a fault?
  - If a user expects "wrong" behavior, is it a fault?

© SERG

Improving Reliability

- **Primary objective:** Remove faults with the most serious consequences.
- **Secondary objective:** Remove faults that are encountered most often by users.

© SERG

Improving Reliability (Cont'd)

- Fixing N% of the faults does not, in general, lead to an N% reliability improvement.
- **<u>90-10 Rule:</u>** 90% of the time you are executing 10% of the code.
- One study showed that removing 60% of software "defects" led to a 3% reliability improvement.

© SERG

The Cost of Reliability

- In general, reliable systems take the slow, steady route:
  - trusted implementation techniques
  - few uses of short-cuts, sneak paths, tricks
  - use of redundancy, run-time checks, type-safe pointers
- Users value reliability highly.
- "It is easier to make a correct program efficient than to make an efficient program correct."

© SERG

The Cost of Reliability (Cont'd)

- Cost of software failure often far outstrips the cost of the original system:
  - data loss
  - down-time
  - cost to fix

© SERG

Measuring Reliability

- Hardware failures are almost always physical failures (*i.e.,* the design is correct).
- Software failures, on the other hand, are due to design faults.
- Hardware reliability metrics are not always appropriate to measure software reliability but that is how they have evolved.

© SERG

Reliability Metrics (POFOD)

- **<u>Probability Of Failure On Demand (POFOD):</u>**
  - Likelihood that system will fail when a request is made.
  - *E.g.,* POFOD of 0.001 means that 1 in 1000 requests may result in failure.
- Any failure is important; doesn't matter how many if > 0
- Relevant for safety-critical systems.

© SERG

Reliability Metrics (ROCOF)

- **<u>Rate Of Occurrence Of Failure (ROCOF):</u>**
  - Frequency of occurrence of failures.
  - *E.g.,* ROCOF of 0.02 means 2 failures are likely in each 100 time units.
- Relevant for transaction processing systems.

© SERG

Reliability Metrics (MTTF)

- **<u>Mean Time To Failure (MTTF):</u>**
  - Measure of time between failures.
  - *E.g.,* MTTF of 500 means an average of 500 time units passes between failures.
- Relevant for systems with long transactions.

© SERG

Reliability Metrics (Availability)

- **<u>Availability:</u>**
  - Measure of how likely a system is available for use, taking in to account repairs and other down-time.
  - *E.g.,* Availability of .998 means that system is available 998 out of 1000 time units.
- Relevant for continuously running systems.
  - *E.g.,* telephone switching systems.

© SERG

Time Units

- What is an appropriate time unit?
- Some examples:
  - Raw execution time, for non-stop real-time systems.
  - Number of transactions, for transaction-based systems.

© SERG

Types of Failures

- Not all failures are equal in their seriousness:
  - Transient vs permanent
  - Recoverable vs non-recoverable
  - Corrupting vs non-corrupting
- Consequences of failure:
  - Malformed HTML document.
  - Inode table trashed.
  - Incorrect radiation dosage reported.
  - Incorrect radiation dosage given!

© SERG

Steps to a Reliability Specification

- For each subsystem, analyze the consequences of possible system failures.
- Partition the failures into appropriate classes.
- For each failure identified, describe the acceptable reliability using an appropriate metric.

© SERG

Automatic Bank Teller Example

- Bank has 1000 machines; each machine in the network is used 300 times per day.
- Lifetime of software release is 2 years.
- Therefore, there are about 300,000 database transactions per day, and each machine handles about 200,000 transactions over the 2 years.

© SERG

Example Reliability Specification

Failure class	Example	Reliability metric
Permanent, non-corrupting	The system fails to operate with any card; must be restarted.	ROCOF =1 occ./1000 days
Transient, non-corrupting	The magnetic strip on an undamaged card cannot be read.	POFOD = 1 in 1000 trans.
Transient, corrupting	A pattern of transactions across the network causes DB corruption.	Should never happen

© SERG

Validating the Specification

- It's often impossible to empirically validate high-reliability specifications.
  - No database corruption would mean POFOD < 1 in 200 million
  - If a transaction takes one second, then it would take 3.5 days to simulate one day's transactions.
- It would take longer than the system's lifespan to test it for reliability!

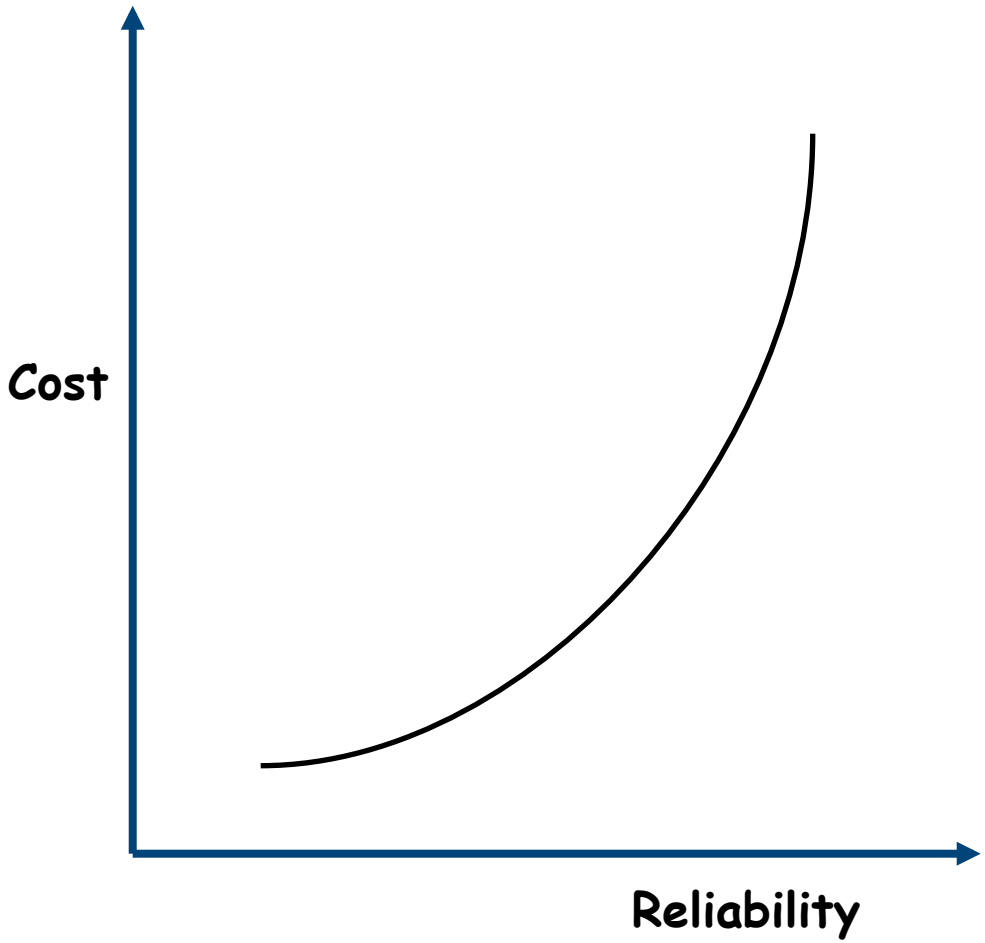
© SERG

Validating the Specification (Cont'd)

- It may be more economic to accept unreliability and pay for failure costs.
- However, what constitutes an acceptable risk depends on nature of system, politics, reputation, ...

© SERG

Increasing Cost of Reliability



© SERG

Steps in a Statistical Testing Process

- Determine the desired levels of reliability for the system.
- Generate substantial test input data based on predicted usage of system.
- Run the tests and measure the number of errors encountered, and the amount of "time" between each failure.
- If levels are unacceptable, go back and repair some faults.
- Once a statistically-valid number of tests have been run with acceptable results, you're done.

© SERG

Problems with Statistical Testing

- Generating "typical" test data is difficult, time consuming, and expensive, especially if the system is new.
- Idea of what constitutes "acceptable" failure rate may be hard to determine:
  - sometimes, expectations are unrealistic.
- Some ideas may be hard to measure:
  - they're hard to specify concretely.
  - small sample size implies results not statistically valid.

© SERG

Programming for Reliability

- As we have seen, squeezing the last few bugs out of a system can be very costly.
  - For systems that require high reliability, this may still be a necessity.
  - For most other systems, eventually you give up looking for faults and ship it.
- We will now consider several methods for dealing with software faults:
  - Fault avoidance
  - Fault detection
  - Fault tolerance, recovery and repair

© SERG

Fault Avoidance

- The basic idea is that if you are REALLY careful as you develop the software system, no faults will creep in.
- Can be done in degrees:
  - **Basic fault avoidance:**
    - Use of information-hiding, strong typing, good engineering principles.
  - **Fault-free software development:**
    - Use of formal specification, code verification, strictly followed software development process.

© SERG

Basic Fault Avoidance

- Basically, you use all of the "holy" ideas we have discussed in class:
  - Programming techniques:
    - information-hiding, modularity, interfaces
    - object-oriented techniques (where appropriate)
    - "structured programming": top-down design, no GOTOs, ...
  - System structure:
    - scrutable system structure
    - simple interactions between components
    - low coupling and high cohesion

© SERG

Basic Fault Avoidance (Cont'd)

- Use of languages and tools that give good support to these ideas!
- No cheats, hacks, or other "risky behavior".

© SERG

"High-Risk Behavior"

- Magic numbers *vs* declared constants.
- Type cheats.
- Compiler quirks, bit ordering, (hardware) architectural dependencies.
- Use of "special knowledge"
  - *E.g.,* implicit shared context between software components
- Features that are likely not to be portable or robust, or make system evolution difficult.

© SERG

Error-Prone Programming Constructs

- Raw pointers:
  - are a notorious cause of crashes,
  - make code incomprehensible to others,
  - it's difficult to make them truly safe.
  - On the other hand, object-oriented programming languages use typed, safe pointers to refer to abstract programming entities.

© SERG

Error-Prone Programming Constructs (Cont'd)

- Dynamic memory allocation:
  - In C, another major cause of crashes and confusion, *e.g.,* ever forget to *malloc()?*
  - Again, object-oriented programming basically solves this.
- Parallelism/concurrency:
  - Logic is hard to get correct.

© SERG

Error-Prone Programming Constructs (Cont'd)

- Floating-point numbers:
  - Many possible problems; may get unexpected results from comparisons leading to unexpected execution paths.
- Recursion:
  - Again, logic is often hard to get completely correct. Also, infinite recursions exhaust memory.

© SERG

Error-Prone Programming Constructs (Cont'd)

- Interrupts:
  - Can make logic of program difficult to follow.
- This isn't to say you should never use these constructs, just that there are real and inherent risks that tend to introduce faults and decrease program reliability.

© SERG

Fault-Free Software Development

- Uses formal specifications, logic, & related tools.
- Extensive and frequent *reviews* of all software artifacts.
- Requires a programming language with strong typing and run-time checking, plus good abstraction mechanisms.
- Requires serious time and effort.

© SERG

Cost of Fault-Free Software Development

- The cost of this approach can be very high!
  - Must have management buy-in on costs.
  - Developers must be experienced and highly trained, not only in traditional software development techniques, but also in mathematics, logic, and special tools.
- Usually, the cost is so prohibitive that it is simple not feasible.

© SERG

Cleanroom Software Development

- Analogy to hardware manufacturing: avoid introducing faults in the first place by operating in a dedicated "clean" environment.
  - What may and may not be used is rigidly enforced.
  - Well-defined process.
  - Attention to minute details.

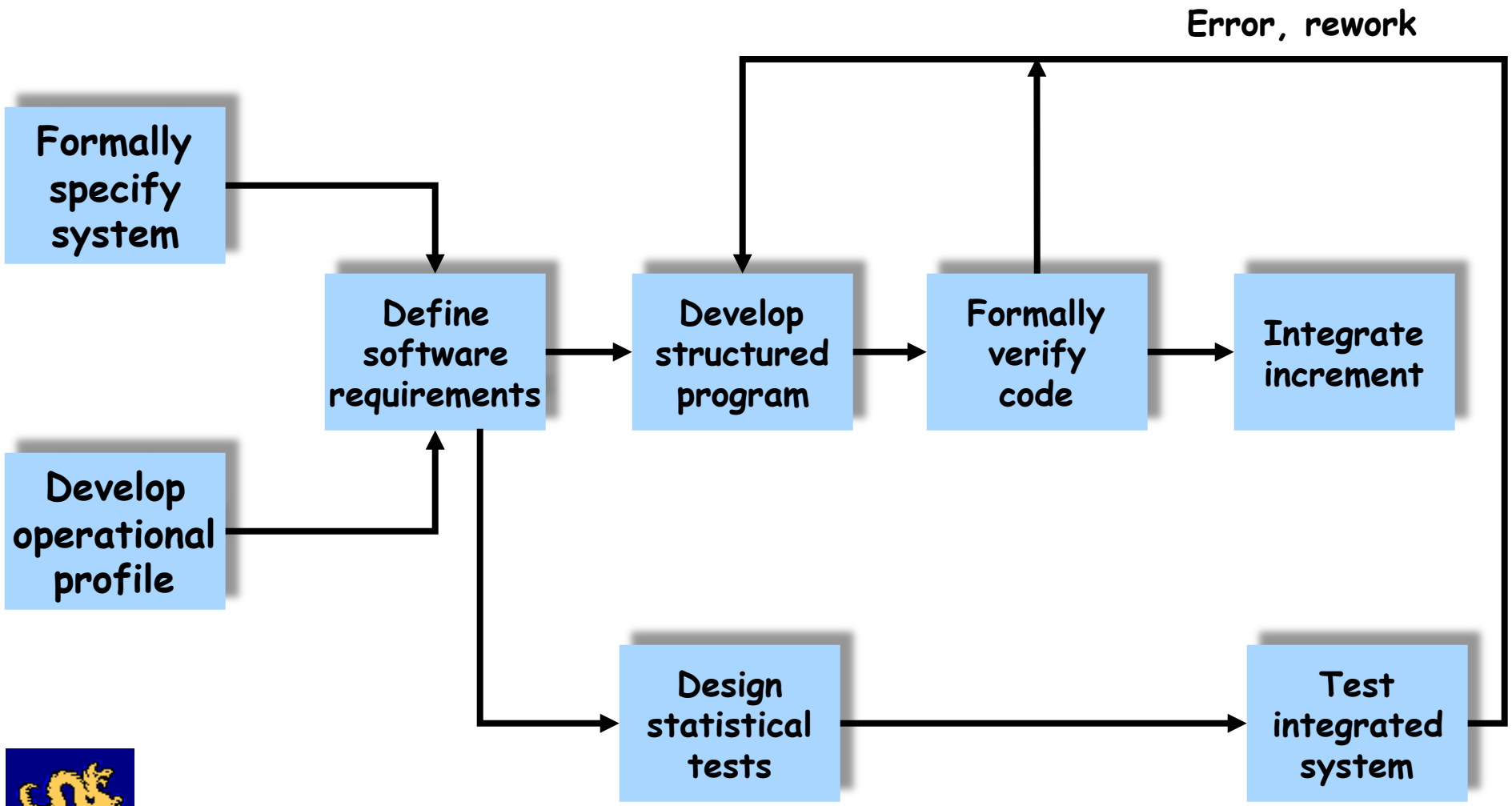
© SERG

Cleanroom Software Development (Cont'd)

- Some impressive (but limited) results:
  - Specialized projects done by IBM, academia.
  - Claim is very high reliability at roughly the same "cost".
- Not clear this will work in industry:
  - Requires highly-trained and patient developers.
  - Potentially costly in time and human resources.

© SERG

Cleanroom Development Process



© SERG

Cleanroom Process is Based on ...

- incremental development
- formal specification
- static verification based on correctness arguments
- statistical testing to determine reliability

© SERG

Cleanroom Process Teams

- **<u>Specification Team:</u>**
  - Responsible for developing and maintaining system specification.
  - Produces a *formal specification* of the system requirements for use by the ...

© SERG

Cleanroom Process Teams (Cont'd)

- **<u>Development Team:</u>**
  - Develops the software based on formal specification provided.
  - Only allowed to use a handful of trusted implementation techniques.
  - Code may be type-checked by tools, but *no executables are generated*.
  - Once code is written, it is *formally verified* against the specification. (static verification)
    - *E.g.,* all loops are guaranteed to terminate.
- Once done, pass code off to ...

© SERG

Cleanroom Process Teams (Cont'd)

- **<u>Certification Team:</u>**
  - While development team is ongoing, the certification team is working in parallel to devise a set of statistical tests to "exercise" the eventual compiled code.
  - Statistical modeling determines when to stop testing.

© SERG

Fault Tolerance

- It is not enough for reliable systems to avoid faults, they must be able to tolerate faults.
- Faults occur for many reasons:
  - Incorrect requirements.
  - Incorrect implementation of requirements.
  - Unforeseen situations.
- Roughly speaking, fault tolerance means *"able to continue operation in spite of software failure."*

© SERG

Steps to Fault Tolerance

- **Detection of failure:**
  - How can you determine if there is a fault?
  - Can we determine what fault caused the failure?
- **Damage assessment:**
  - Which parts of the system have been affected?
  - How serious is the damage?
  - Can we ignore it? Fix it?

© SERG

Steps to Fault Tolerance (Cont'd)

- **<u>Fault recovery:</u>**
  - Restore the program state to a previous "safe" state.
  - *E.g.,* Restoring a database and unfolding the most recent transactions.
  - May require "snapshots" of old states to be kept.

OR

© SERG

Steps to Fault Tolerance (Cont'd)

- **<u>Fault repair:</u>**
  - Can you simply fix the damage?
  - Can you stop the fault from occurring again? without shutting down the system?
  - *E.g.,* mail router: just try a different route

© SERG

Software Fault Tolerance Programming Techniques

- N-Version programming (NVP)
- Exception Handling
- Subtypes
- Run-time Assertions

© SERG

N-Version Programming (NVP)

- Basic idea from hardware TMR: use three identical components that vote; majority rules.
- With NVP, have three distinct programs that vote.
- Claimed success has been criticized in some circles.

© SERG

NVP Criticism

- Software failure is a design fault, not a mechanical failure.
- How do you know which team interpreted the requirements correctly?
- This technique has been used in the real world!
  - The Airbus-320 uses NVP in its on-board software.

© SERG

Exception Handling

- Sometime during execution you detect that some kind of failure has occurred (*i.e.*, "an exception has been raised").
- While you could repair it in-line, usually these kinds of failures can occur in multiple places.
- It makes sense to define common handling routines in one place.

© SERG

Exception Handling (Cont'd)

- An exception handler is a specialized routine for dealing with an error. Usually, handlers are grouped together.
- Both C++ and Java support exception handling (in roughly the same manner).
  - Java provides a variety of predefined exceptions; C++ provides none.

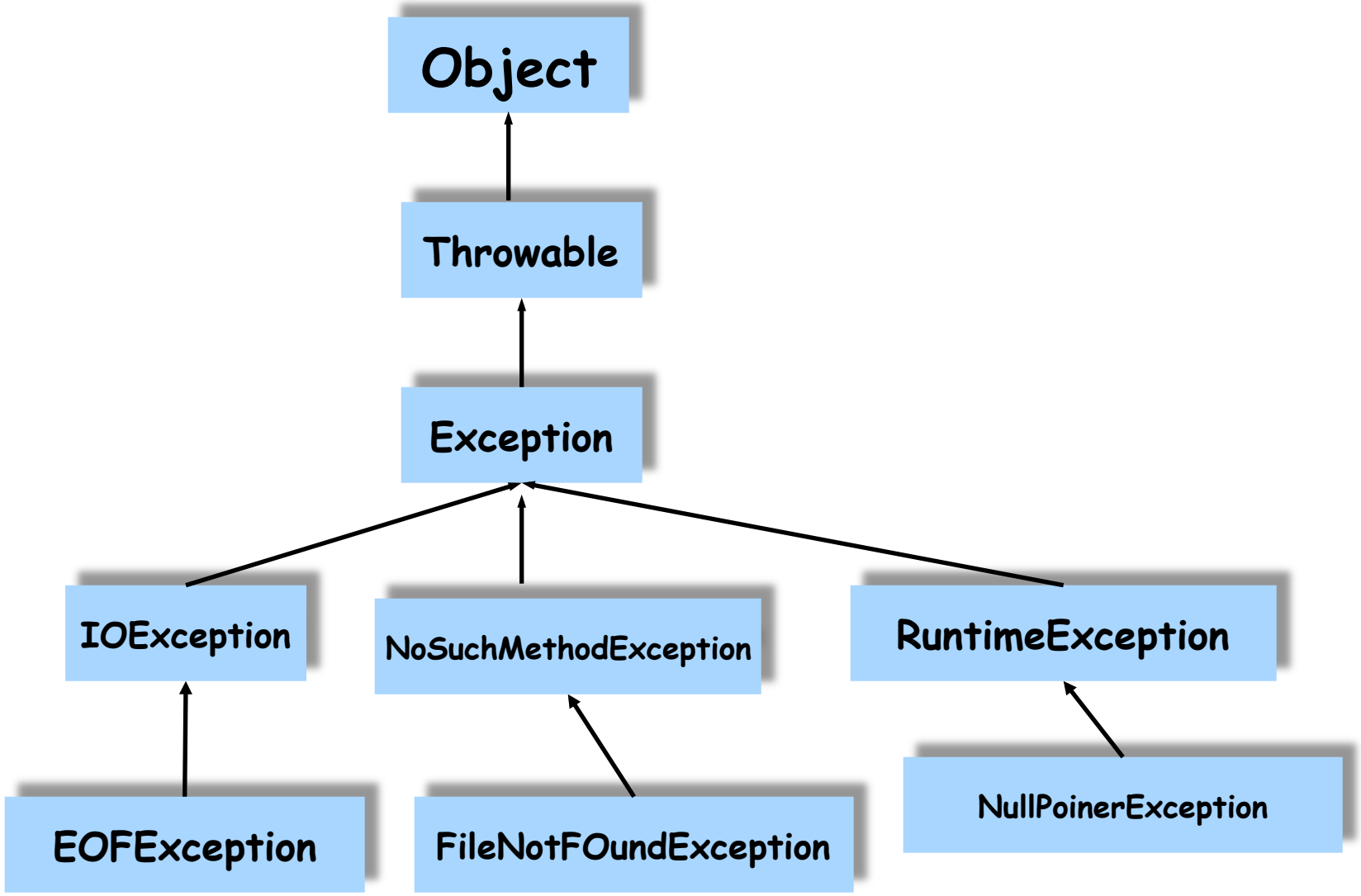
© SERG

Exception Handling (Cont'd)

- We'll assume the Java model; C++ is similar.
- Exceptions are objects.
- The only interesting thing about them is the name of the defining class and its place in the overall exception class hierarchy:

© SERG

Java Exception Class Hierarchy



Exception Handling in Java

```
public static String get () throws EOFException {
    String ans;
    if (tokenizer == null || !tokenizer.hasMoreTokens()) {
        try {
            curLine = inStream.readLine ();
            tokenizer = new StringTokenizer(curLine);
        }
        catch (IOException e) {
            if (!(e instanceof EOFException)) {
                put ("Error: " + e.getMessage());
            }
        }
    }
    if (tokenizer.hasMoreTokens()) {
        return tokenizer.nextToken();
    }
    else {
        throw new java.io.EOFException();
    }
}
```


© SERG

Exception Handling in Java

- In Java and C++, you raise an exception explicitly via a **throw** statement.
- The function that throws the exception may choose to handle it locally; more likely, it will let the caller decide what to do.
- If the caller handles the exception completely, then all is fine.
- However, it is common to handle only some cases, and then let the caller of the caller handle the rest.

© SERG

Exception Handling in Java (Cont'd)

- Any function that raises an exception (either explicitly via a **throw** or implicitly by calling another function that raises an exception) must either:
  - Explicitly handle all possible exceptions.
  - Declare itself as (possibly) raising an exception.

© SERG

Exception Handling in Java (Cont'd)

- If you have a block of code that might raise an exception, embed it in a **try** block.
- Right after the **try** block, you can try to resolve those exceptions by a sequence of **catch** statements.
- If the exception does not get resolved by one of these **catch'**s, then the exception is sent back to the caller for resolution.

© SERG

Exception Handling in Java (Cont'd)

- Use of inheritance to structure exception classes is very intuitive and useful.
- Exceptions often don't have a very interesting state.
  - Usually, there's an error message,
  - Perhaps some parameters to indicate what/ where things went wrong.

© SERG

Exception Handling in Java (Cont'd)

- Exception handlers are a way to separate out the basic functionality from the exceptional conditions.
- The old-fashioned way would be to pass various status flags with every function call.

© SERG

Exception Handling in Java (Cont'd)

- Try to limit exceptions to exceptional circumstances and real error conditions.
- Expected behavior (such as normal EOF) may be better handled in-line, if you have a choice.
- The basic Java libraries make extensive use of exceptions, so it's hard to avoid them.
- Yes, exceptions usually entail some overhead. Most feel it is a worthwhile tradeoff.

© SERG

Other Software Fault Tolerance Programming Techniques

- **<u>Subtypes:</u>** *E.g.,*
  - integer sub-ranges: 1..10,
  - even integers, > 0,
  - proper enumerated types, subclasses
- **<u>Run-time Assertions:</u>** *E.g.,*
  - Module/class invariants, pre/post-conditions
- All of these must be enforceable by language or system to be practically useful.

© SERG

Cost of Using Subtypes & Assertions

- The performance overhead may be significant.
- Might want to disable run-time checks after sufficient testing.
- *E.g.,*
  - Anna is a superset of Ada;
  - Anna adds special run-time checks embedded in what the Ada compiler thinks are comments.

© SERG

Damage Assessment

- Once a failure has been detected, must try to analyze extent of damage:
  - What parts of the state have been infected?
  - Is there any built-in redundancy to check for validity?
  - Are values within legal range?
  - Are invariants preserved?

© SERG

Common Assessment Techniques

- **Checksums** are used in data transmission (*e.g.*, parity)
- **Redundant pointers** check validity of data structures
- **Watchdog timers:** If process doesn't respond after a certain period, check for problems.

© SERG

Forward Fault Recovery

- Try to fix what's broken based on understanding of program and then carry on.
- Often requires redundancy in representation.
- If you can figure out which value is obviously wrong, you may be able to derive its real value from the rest of the state.

© SERG

For Example ...

- Data transmission techniques.
- Redundant pointers
  - If enough pointers are OK, can rebuild structures such as file system or database.
- Sometimes, domain understanding is enough to help figure out what "real" values should be.

© SERG

Backward Fault Recovery

- Restore state to a known safe state.
  - *E.g.,* Restore file system from tape backup.
- Some systems are *transaction oriented*:
  - Changes are not committed until computation is complete.
  - If an error is detected, changes are not applied.
- Requires period "snapshots" to be taken and maintained.
- Simple approach, but may entail loss of data.

© SERG